



# **PROCESADORES DE LENGUAJE - trad2**

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

## 1. Introducción

En esta segunda sesión se ha completado el desarrollo del traductor de fragmentos en lenguaje C a notación prefija tipo Lisp. Partiendo de la base de la práctica anterior, se han incorporado nuevas funcionalidades que permiten la traducción de operadores lógicos, operadores relacionales adicionales y funciones de salida más realistas, como `puts` y `printf`.

El objetivo ha sido alcanzar una cobertura más completa de un subconjunto de C, respetando el formato funcional de Lisp, sin alterar el analizador léxico original (`yyllex()`), e integrando estas extensiones de forma modular y natural en la gramática Bison.

## 2. Descripción del Código

El archivo `trad2.y` amplía la gramática anterior, reorganizando las reglas de producción para mejorar la legibilidad y mantener la compatibilidad con la entrada esperada en C. A continuación se resumen las principales mejoras:

- **Separación de instrucciones simples y bloques:** se ha diferenciado entre instrucciones que requieren punto y coma (`;`) y aquellas que no lo necesitan (como `if`, `while`), eliminando la obligatoriedad de usar `;` tras los bloques de control.
- **Operadores relacionales adicionales:** se han añadido las comparaciones `==`, `!=`, `<=`, `>=`, que se traducen respectivamente como `(= ...)`, `(/= ...)`, `(<= ...)`, `(>= ...)`.
- **Operadores lógicos:** se han implementado `&&`, `||` y `!` como `(and ...)`, `(or ...)` y `(not ...)`, respetando la precedencia y asociatividad habitual del lenguaje C.
- **Traducción de `puts` y `printf`:**
  - `puts("texto")` se traduce como `(print "texto")`.
  - `printf("texto", x, "otra", y)` se traduce como múltiples `(princ ...)`, conservando el orden.
- **Gestión de expresiones anidadas:** se ha verificado que las expresiones con múltiples niveles de paréntesis se traducen correctamente.
- **Restricción del analizador léxico:** dado que no se permite modificar `yyllex()`, se ha adaptado la gramática para reconocer el carácter `'!'` directamente como operador unario, sin necesidad de un token `NOT`.

### 3. Casos de Prueba

Se ha preparado un fichero de prueba completo que cubre todas las estructuras exigidas en el enunciado:

```
C/C++  
  
int a = 3, b, c = 0;  
  
main() {  
    puts("INICIO");  
    b = 5;  
    c = a + b * 2;  
  
    if (c >= 10 && !(a == 4)) {  
        puts("CONDICION 1 OK");  
    } else {  
        puts("CONDICION 1 FALLA");  
    }  
  
    if (a != b || c < 15) {  
        printf("Resultado: ", c);  
    }  
  
    while (c > 0) {  
        printf("c vale ", c);  
        c = c - 1;  
    }  
  
    puts("FIN");  
}
```

Salida generada:

```
Unset  
  
(setq a 3)  
(setq b 0)  
(setq c 0)  
(defun main ()  
  (print "INICIO")  
  (setq b 5)  
  (setq c (+ a (* b 2)))  
  (if (and (>= c 10) (not (= a 4)))  
      (print "CONDICION 1 OK")  
      (print "CONDICION 1 FALLA"))  
)
```

```
)  
(if (or (/= a b) (< c 15))  
  (princ "Resultado: ")  
  (princ c)  
)  
(loop while (> c 0) do  
  (princ "c vale ")  
  (princ c)  
  (setq c (- c 1))  
)  
(print "FIN")  
)  
(main)
```

## 4. Conclusiones

Con esta ampliación se ha completado un traductor funcional desde un subconjunto representativo de C hacia notación Lisp. Se han abordado y superado limitaciones técnicas impuestas por el analizador léxico fijo, y se ha mejorado la usabilidad del lenguaje de entrada al eliminar la necesidad de punto y coma tras bloques de control.

Entre los avances más relevantes:

- Incorporación de lógica booleana compleja (&&, ||, !)
- Gestión de expresiones condicionales con operadores compuestos
- Traducción más realista de salida (printf)
- Diseño limpio y extensible de la gramática